PERTURBING INPUTS IMPROVES PRETRAINING DISTILLATION

Ziyang Cai[1] **Yihe Dong**[1] **Dimitris Papailiopoulos**[2] **Shivam Garg**[2]
[1]Princeton University [2]Microsoft Research
{zc5794,ydong}@princeton.edu, {dimitriosp,shigarg}@microsoft.com

ABSTRACT

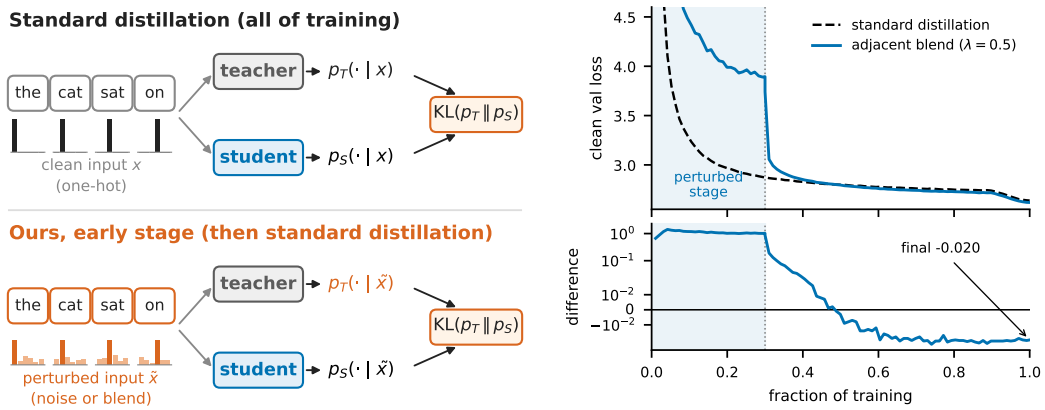
Knowledge distillation trains a student language model to match a teacher's predictions on the training text. We ask whether learning the teacher's behavior beyond the training text can further improve the student. We investigate this using three input perturbations: additive token noise, blending two documents, and blending neighboring tokens. Specifically, for an early part of pretraining we perturb the inputs and train the student to match the teacher on them, and we spend the rest of the same token budget on ordinary distillation. When pretraining from scratch, the best method improves on standard distillation at both scales we tried, by 0.025 nats at 135M parameters and by 0.020 nats at 0.5B (mean of three seeds). At 135M this is 1.45 times the gain that distillation itself provides over next-token training. We also observe promising downstream transfer: at 0.5B, the best variant improves average zero-shot accuracy by about one percentage point across seven tasks. It also retains its validation-loss advantage after low-bit weight quantization. We also test a data-limited setting in which training data must be reused, and find that the gains persist. Finally, ablations suggest that, for additive noise, perturbing the teacher's inputs contributes most of the gain, while perturbing only the student's inputs yields much less.

1 INTRODUCTION

When pretraining small language models, it is common to use distillation from a larger model (Hinton et al., 2015; Gemma Team, 2024; Peng et al., 2025; Muse Glimmer Team, 2026). The student learns to match the teacher's next-token distribution, but only on the training text itself. The teacher defines a prediction function beyond those inputs, and its behavior elsewhere is information the student never receives. In this work, we ask whether learning some of that behavior can improve pretraining distillation.

This question connects naturally to data augmentation. In language-model pretraining, augmentation typically corrupts the student's input, for example by masking or replacing tokens, while keeping the original next token as the target (Xu et al., 2026; Chen et al., 2026; Ni et al., 2025). A teacher makes a different kind of augmentation possible: it can predict on the corrupted input itself, providing a target that reflects what the student actually sees. Training on teacher-labeled augmented inputs is established in task-specific distillation of language models (Jiao et al., 2020; Rashid et al., 2021) and in vision (Beyer et al., 2022; Wang et al., 2022). Whether it helps during pretraining from scratch is less clear, because training often proceeds without repeating the data, and the teacher's predictions on clean text already provide rich supervision.

We investigate this question in pretraining from scratch through a two-stage distillation schedule (Section 2, Fig. 1). Specifically, we study three perturbations: additive token noise, blending two documents position by position, and blending neighboring tokens into a single input position. Teacher and student receive perturbed inputs during an early stage of training, followed by ordinary distillation on clean inputs. All three perturbations improve on standard distillation at both student scales we study (Sections 3–4). Adjacent-token blending is the strongest variant, reducing validation loss by 0.025 nats at 135M parameters and 0.020 nats at 0.5B, with the same number of training steps as standard distillation. To put this improvement in context, at 135M the additional gain is 1.45 times the gain that distillation itself provides over next-token training.



(a) Standard distillation and our perturbed stage

(b) Two-stage schedule (0.5B)

Figure 1: **Overview.** (a) In standard distillation, teacher and student read the same clean text and the student is trained to match the teacher's prediction on it. In the early stage of our method, both read a perturbed version of the text, and the student matches the teacher's prediction on that perturbed input: the inputs on which the teacher supervises the student change, not only the student's input. The bars under each token sketch its input vector over a toy vocabulary, one-hot for clean text and spread over several vocabulary items when perturbed (Fig. 2). The rest of training is standard distillation on clean text. (b) Clean validation loss of two 0.5B students trained from the same initialization on the same data order, one with standard distillation and one with adjacent blending (λ=0.5) for the first 30% of training, and their difference (symmetric log scale). While its inputs are perturbed, the blended student lags by about one nat. After the switch it recovers quickly, overtakes the baseline at about 47% of training and finishes 0.020 nats below it.

To understand the source of these gains, we run several ablations (Section 5). Without a teacher, perturbing the inputs while training the student to predict the original next token provides little benefit or is sometimes harmful. With additive noise, we also perturb only the teacher: the student sees clean text but learns to match the teacher's prediction on a perturbed version of it. This achieves about 90% of the gain obtained when both models are perturbed. In contrast, perturbing only the student while keeping the teacher's input clean achieves about a third. Thus, teacher-side perturbation contributes most of the improvement. Further ablations find that averaging four noisy teacher predictions gives a smaller improvement than using a single draw, despite their identical expected targets (Appendix D). These ablations suggest a plausible explanation: noisy teacher targets make some fine-grained, unstable details of the teacher's predictions harder to fit precisely during training. This may encourage the student to learn the teacher's more stable patterns without having to reproduce every detail of its predictions on clean text.

We also study data-limited pretraining, where training repeatedly uses the same text and regularization becomes especially important (Section 6). With 16 epochs over 262M unique tokens, additive noise still improves distillation by around 0.020–0.023 nats across two seeds, showing that the benefits persist beyond the single-pass setting.

Finally, we evaluate downstream transfer and post-training weight quantization (Section 7). At 135M, changes in average zero-shot accuracy are small, at most 0.6 points. At the larger 0.5B scale, the best method increases average zero-shot accuracy by about one percentage point across seven tasks. It also retains its validation-loss advantage after low-bit weight quantization. Together, these results suggest that perturbed-input distillation can improve the downstream utility of compact students, rather than only their full-precision validation loss.

Our main contributions are:

- A proof of concept: learning a teacher's behavior beyond ordinary token inputs can improve language-model pretraining distillation.

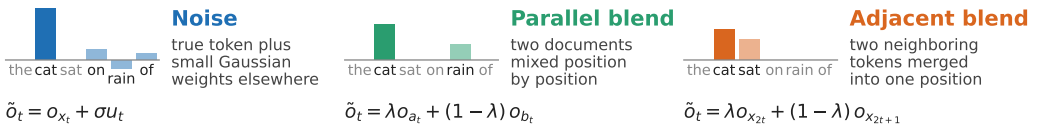


Figure 2: **Perturbation families** on a toy vocabulary. Each replaces the one-hot vector of a token with a sparse weighted combination of vocabulary items, with the position's own token shown dark. Teacher and student both map the perturbed vector through their own embedding tables, and the student is trained to match the frozen teacher's next-token distribution on it.

- An investigation of the source of the gains: ablations separate teacher-side and student-side effects and probe the role of variability in the teacher's targets.
- Benefits beyond the main training setting: improvements persist under data repetition with tuned regularization, transfer to zero-shot tasks, and survive low-bit weight quantization.

2 Method

One-hot perturbations (Fig. 2). Let $o_v \in \mathbb{R}^{|V|}$ be the one-hot vector of vocabulary item v, so position t of a clean sequence x holds o_{x_t}. A perturbation replaces o_{x_t} by a sparse weighted combination $\tilde{o}_t = \sum_j w_j\, o_{v_j}$ of a few vocabulary items. Each model computes its input embedding as $E^\top \tilde{o}_t$ with its own embedding matrix E. Teacher and student receive the same (v_j, w_j), so they see the same perturbation whatever their embedding width. The loss is the forward KL divergence from the teacher's next-token distribution to the student's at every position (temperature 1).

Perturbation families.

- **Noise.** $\tilde{o}_t = o_{x_t} + \sigma u_t$, where u_t places independent Gaussian weights $\mathcal{N}(0, 1/k)$ on k=8 tokens drawn uniformly from the vocabulary (with replacement, so the true token can occasionally be among them), resampled at every position; the weights are not normalized. σ sets the size of the perturbation relative to the true token.
- **Parallel blend.** Two independent documents a, b are mixed position by position, $\tilde{o}_t = \lambda o_{a_t} + (1-\lambda) o_{b_t}$.
- **Adjacent blend.** Consecutive token pairs of one document are merged into one position, $\tilde{o}_t = \lambda o_{x_{2t}} + (1-\lambda) o_{x_{2t+1}}$. Sequences are drawn twice as long, so inputs keep their full length.

The blends also pack two corpus tokens into each model-input position during the perturbed stage, increasing text exposure without increasing the model sequence length or the number of training steps. Outside Section 6, all token budgets are sized so that no token is seen twice. We test whether the extra text matters in Section 5.

Two-stage schedule. Training consists of a perturbed stage covering a fixed fraction of the run, followed by ordinary distillation on clean inputs for the remainder (Fig. 1b). The learning rate follows a warmup–stable–decay schedule (Hu et al., 2024): 10% warmup, a constant plateau spanning the stage boundary, and a 10% final decay, which therefore always takes place on clean inputs. The baseline is the identical run without the perturbed stage.

3 Experimental setup

Models and data. *135M scale:* a SmolLM2-135M architecture trained from random initialization and distilled from SmolLM2-1.7B (Allal et al., 2025), for 8000 steps of 512×1024 tokens (4.2B tokens) on a 58.6B-token FineWeb-Edu corpus (Penedo et al., 2024). *0.5B scale:* a Qwen2.5-0.5B architecture trained from random initialization and distilled from Qwen2.5-1.5B (Qwen Team, 2025), for 19073 steps of 256×2048 tokens (10B tokens) on a 65B-token FineWeb-Edu corpus. No data is repeated at either scale, except in the data-constrained experiments of Section 6.

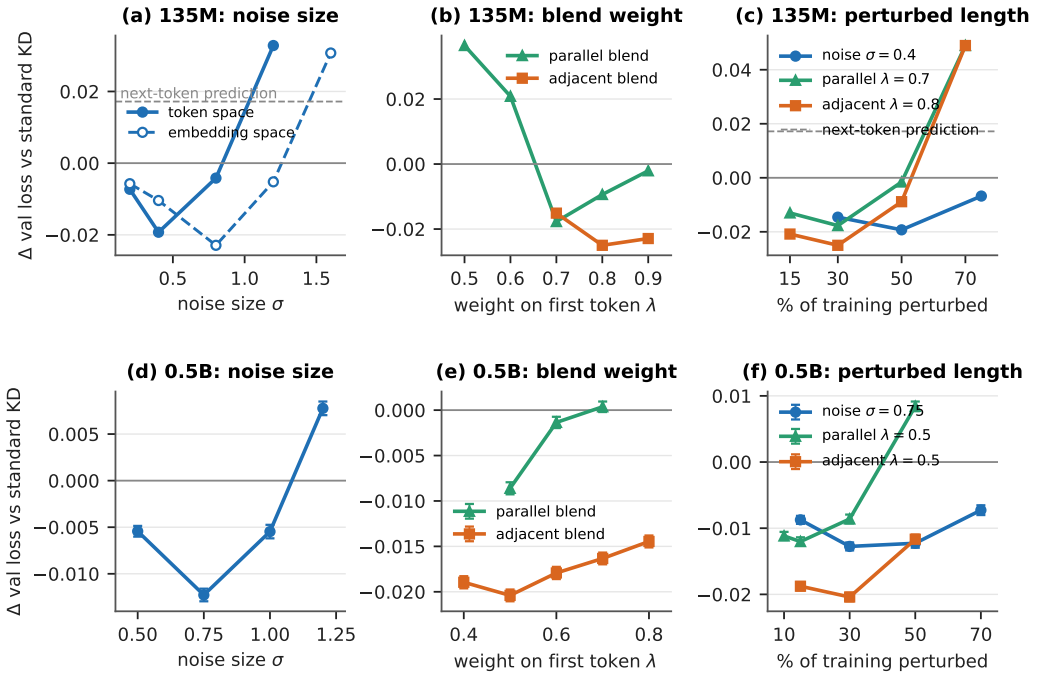


Figure 3: **Change in final validation loss against standard distillation** (below zero is better). (a–c) 135M student with the SmolLM2-1.7B teacher, 4.2B tokens. The dashed line is next-token prediction without a teacher. (a) Noise size, with a 50% perturbed stage. The open markers in (a) are Gaussian noise added to each model's input embedding; their σ is relative to the embedding's root-mean-square value, so it is not in the same units as token noise. (b) Blend weight for both blends, with a 30% perturbed stage. (c) Length of the perturbed stage at each family's best size. (d–f) 0.5B student with the Qwen2.5-1.5B teacher, 10B tokens, scored on the full validation split. Error bars are ± 2 paired standard errors. (d) Noise size and (e) blend weight, with the same perturbed-stage lengths as in (a, b). (f) Length of the perturbed stage at each family's best size.

Paired comparisons. All runs at a scale read the same data in the same order and start from the same initialization, and differ only in the training objective. As a check, runs from the same seed agree on validation loss at the first evaluation to within 0.004 nats. The next-token-prediction reference uses the same data stream.

Evaluation. At 0.5B every final model is scored on the full held-out validation split (2441 non-overlapping windows of 2048 tokens, identical for every run). We report the difference to standard distillation paired per window, with its standard error. At 135M we report the validation loss from the training run's evaluation pass; differences below about 0.003 nats are within its resolution. Most configurations are trained once. The main 135M ablations, the data-constrained comparison and the best 0.5B configuration are repeated with two or three initialization seeds, always on the same data order. For configurations trained once, error bars reflect evaluation sampling rather than variation between training runs.

4 Results

All three perturbations improve on standard distillation at 135M (Fig. 3a–b). For scale, distillation itself improves on next-token prediction by 0.017 nats. Adjacent blending is the strongest variant. At λ=0.8 it lowers the loss by a further 0.025 nats, almost one and a half times what distillation itself gains. Noise at σ=0.4 and parallel blending at λ=0.7 gain 0.019 and 0.018. Every family has a clear optimum, and going well past it hurts: large noise, and parallel blends heavier

than λ=0.7, end worse than standard distillation. Gaussian noise added to the embeddings behaves the same way, with its optimum at a somewhat larger σ in its own units.

Length of the perturbed stage (Fig. 3c, f). For both blends at 135M, a perturbed stage of 30% of training works best. A shorter stage gives less of the gain, and when 70% of training is perturbed both blends end up worse than next-token prediction. Noise is more forgiving. It does best with half of training perturbed and still helps at three quarters. Blended inputs are much further from natural text than small noise, which is presumably why less training on them is useful. The 0.5B runs show the same ordering. Adjacent blending is again best with a 30% stage, noise does about equally well with 30% or 50%, and parallel blending, the heaviest perturbation we use, does best with only 15% and is harmful at 50%.

The gains hold at 0.5B (Fig. 3d–e). With the Qwen2.5-1.5B teacher, adjacent blending is again the strongest variant. An even blend of the two tokens works best and improves on standard distillation by 0.020 nats, with a paired standard error of 0.0003. Two more initialization seeds of this run and its baseline gave 0.021 and 0.019. The gain shrinks steadily as the blend becomes less even. Noise is best at σ=0.75, where it gains 0.012 nats, and parallel blending is again best with an even blend, where it gains 0.009 with a 30% stage and 0.012 with its best, 15% stage. Its gain is gone by λ=0.7, the best parallel blend at 135M. Since the two documents play symmetric roles, λ=0.5 is the heaviest blend this family allows.

Too little or too much noise. At both scales the gain from noise first rises and then falls with its size. Small noise barely changes the teacher's predictions. At 0.5B, σ=0.4 raises the frozen teacher's loss by only 0.07 nats, and σ=0.5 gives less than half the gain of σ=0.75. The teacher's targets on such inputs are close to its targets on clean text, so the student gets little that standard distillation does not already provide. Large noise hurts. At σ=1.2 the teacher's loss rises by 3.5 nats, and the student ends up worse than with standard distillation. The useful range lies in between, where the perturbation changes the teacher's predictions without making them uninformative. We do not have a principled way to locate this range in advance. The best size differs between our two settings, σ=0.4 at 135M and 0.75 at 0.5B, and we found it by sweeping.

5 Ablations

The ablations ask where the gain comes from (Fig. 4). For our perturbations, the answer is the teacher's predictions on perturbed inputs. The gain needs a teacher, and it comes mostly from perturbing the teacher's input rather than the student's. It is not explained by the teacher's targets simply being softer or worse. It also does not depend on two details of our construction, sharing one draw between the two models and perturbing in token space. Nor, for adjacent blending, does it come from the extra text it reads. These ablations identify important factors rather than a complete explanation of the gains. All ablations are at 135M and, unless noted, use token noise with σ=0.4 for the first half of training, the best noise setting. Its gain is 0.019 nats for the seed shown in Fig. 3 and 0.015 averaged over three seeds. Where we have several initialization seeds, bars show the mean and dots the individual seeds, and each seed is compared with standard distillation from the same seed. The gain from noise varies by up to 0.008 nats between seeds, which matters when reading the smaller differences below.

Without a teacher (Fig. 4a). Running the same noise or parallel-blend schedule with ordinary next-token targets and no teacher improves on next-token prediction by only 0.003 nats, about a seventh of the gain with a teacher. For adjacent blending the contrast is starker. Without a teacher, training on the blended inputs ends 0.03 nats behind plain next-token prediction, although the same inputs give the largest gain of all when a teacher labels them.

Teacher side and student side (Fig. 4b). Next we perturb only one model's input. When only the teacher's input is perturbed, the student sees clean text but is trained toward the teacher's prediction on a perturbed version of it. Averaged over three seeds, this recovers about 90% of the gain. When only the student's input is perturbed, with the teacher's prediction on the clean text as the target, the gain falls to about a third, not much more than the same noise gives without any teacher. This

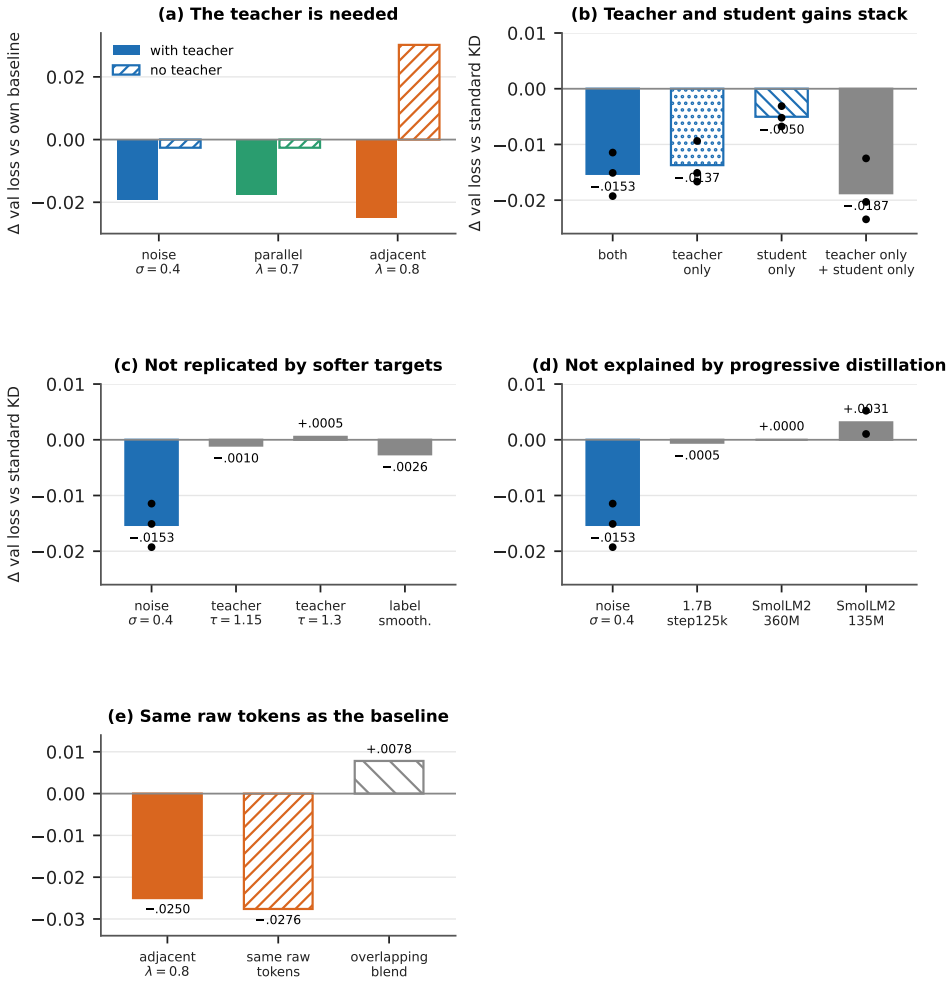


Figure 4: **Ablations** (135M, SmolLM2-1.7B teacher; difference in final validation loss, lower is better). Bars show the mean over initialization seeds and dots the individual seeds, each compared with standard distillation from the same seed. (a) Each family at its best setting, with and without the teacher, each compared with its own unperturbed baseline (standard distillation or next-token prediction). (b) Token noise given to both models, to the teacher only, or to the student only. The grey bar is the sum of the two one-sided gains. (c) Clean inputs, with the teacher's target softened instead, either by a temperature τ (τ=1.15 matches the entropy of the teacher's prediction on σ=0.4 inputs) or by label smoothing to the same entropy. (d) Clean inputs for the first half of training, distilled from a weaker model (an early checkpoint of the teacher, or a smaller model) instead of the teacher. (e) Adjacent blending at λ=0.8, as in the main runs and on the same raw tokens as the baseline, and the overlapping variant described in the text (one seed).

student-only arm is the usual form of data augmentation for language models, which perturbs the input and keeps the target (Xu et al., 2026; Chen et al., 2026). The two one-sided gains add up to about a fifth more than the gain with both sides perturbed, so the two effects combine roughly additively. Appendix C compares random-token replacement, the augmentation used in concurrent work on data-constrained pretraining.

Softer targets (Fig. 4c). A perturbed input makes the teacher less certain. At σ=0.4 the entropy of its prediction rises from 1.97 to 2.46 nats, so the gain might come from a flatter target alone. It does not. Softening the teacher's prediction on clean inputs to the same entropy, with a temperature or with label smoothing, gains at most 0.003 nats, and a stronger temperature gains nothing. Applying

the usual distillation temperature to both models, which reweights the loss without changing its optimum, is slightly harmful.

A weaker early teacher (Fig. 4d). A perturbed teacher is also a worse predictor, and distilling first from a weaker or partly trained teacher can act as a curriculum (Panigrahi et al., 2025). To test this, we distill the first half of training on clean inputs from a weaker model instead: SmolLM2-135M, whose loss is close to that of the 1.7B teacher on $\sigma{=}0.4$ inputs, SmolLM2-360M, or the earliest published checkpoint of the 1.7B teacher. None of them helps. All three end within 0.003 nats of standard distillation.

Shared draws and token space are not required. Drawing the teacher's and the student's noise independently gives the same gain as a shared draw (Appendix D), and Gaussian noise added to each model's input embedding reaches a similar gain at its own best size (Fig. 3a). The one-hot construction is a convenient way to define one perturbation for models of different width, but the gain does not come from it. What the method needs is a teacher that predicts on the perturbed input. Relatedly, in teacher-only experiments, averaging four noisy teacher predictions gives a smaller gain than using one draw (Appendix D). A plausible explanation is that noise encourages the student to focus on what the teacher predicts consistently across perturbed inputs rather than fitting details that change from one perturbation to another, and averaging reduces that variability without changing the expected target.

The extra text is not the source of adjacent blending's gain (Fig. 4e). Because adjacent blending packs two corpus tokens into each input position, its perturbed stage reads twice as much text as standard distillation does in the same steps. To remove this difference, we rerun adjacent blending at $\lambda{=}0.8$ on exactly the tokens the baseline reads in that stage, packed into half as many positions, so the blended student sees no extra text and also gets less training signal per step. On the same seed it gains 0.028 nats, slightly more than the 0.025 of the regular run. We also tried an overlapping variant, in which each position blends a token with the one after it, so that the student again reads exactly the baseline's text at every position. It ends 0.008 nats worse than standard distillation, but it changes the task as well, because each input then contains part of the token the model is asked to predict.

6 Data-constrained pretraining

When unique data are limited, training repeats them for several epochs, and regularization becomes important: properly tuned weight decay changes conclusions (Kim et al., 2026; Xu et al., 2026), and input corruption with clean targets helps next-token training (Xu et al., 2026; Chen et al., 2026; Ni et al., 2025). We test whether the gain of distilling on perturbed inputs survives this setting and a tuned baseline. At 135M, with the same 4.2B-token budget, training repeats the first 262M tokens of the corpus for 16 epochs. The tokens are cut into non-overlapping 1024-token windows, reshuffled each epoch with the same shuffle for every run. We sweep AdamW weight decay over 0.1, 0.3, 1, 3 and 10 for standard distillation, of which 10 diverges, and use token noise $\sigma{=}0.4$ for the first 50% of training (Fig. 5).

Gain on a tuned baseline. Weight decay matters under repetition. Raising it from the default of 0.1 to 1.0, the best value for standard distillation, lowers that baseline's loss by 0.033 nats. On this tuned baseline, distilling on perturbed inputs still gains 0.023 and 0.020 nats for the two seeds, while standard distillation itself differs by only 0.001 between them, and noise does not shift the best weight decay. The same noise without a teacher gains 0.008, so the teacher makes the gain about 2.6 times larger. At the default weight decay the gain from noise looks larger, 0.032, but much of that is regularization that a tuned weight decay also provides. On the tuned baseline the gain is about the same as without any repetition: with fresh data and the same weight decay of 1.0, noise gains 0.020 nats. A weight decay of 1.0 also helps standard distillation on fresh data, by 0.005 nats, so our default baseline is slightly under-tuned, but the gains do not shrink on the tuned one. Adjacent blending, in a single run, gains 0.035 there, compared with 0.025 at the default weight decay. With milder repetition, 4 epochs over 1.05B unique tokens, repetition costs standard distillation only 0.006 nats and the gain is 0.023.

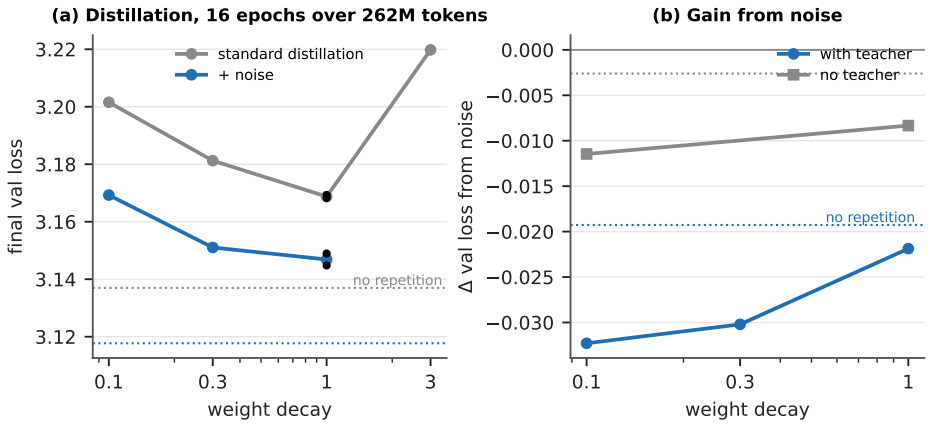


Figure 5: **Data-constrained pretraining** at 135M: 16 epochs over 262M unique tokens, same 4.2B-token budget. (a) Final validation loss of standard distillation with and without noise (token noise σ=0.4 for the first 50% of training) as a function of AdamW weight decay. Black dots are two initialization seeds at weight decay 1, and dotted lines show the same 4.2B-token budget without repeated data (weight decay 0.1). Weight decay 10 diverges. (b) The gain from the same noise with and without a teacher. Dotted lines again show the gain without repeated data. Exact values in Table 4 (Appendix B).

Table 1: **Increase in validation loss from post-training weight quantization** at 0.5B (nats; 512 fixed validation windows, paired across models). RTN: round-to-nearest; GPTQ and AWQ as implemented in llm-compressor. All linear layers except the output head are quantized; activations stay in 16 bits. Perturbed stage: 30%, or 15% for the parallel blend. Bold: better than standard KD.

	full	int4, group 128			int3, group 128		
	precision	RTN	GPTQ	AWQ	RTN	GPTQ	AWQ
standard KD	2.6782	0.205	0.028	0.019	6.226	0.160	0.099
noise σ=0.75	**2.6642**	0.317	**0.027**	0.019	**5.623**	**0.117**	**0.097**
parallel λ=0.5	**2.6660**	**0.161**	**0.024**	0.020	**1.480**	**0.123**	0.099
adjacent λ=0.5	**2.6579**	**0.113**	**0.024**	0.019	**1.203**	**0.120**	**0.098**

7 Quantization and downstream evaluation

We evaluate the final students, standard distillation and each family's best setting, after post-training weight quantization at 0.5B (Table 1) and on zero-shot benchmarks at both scales (Table 2). Quantization damage is the increase in validation loss on 512 fixed validation windows, paired across models; all linear layers except the output head are quantized to int4 or int3 with groups of 128 weights. We compare round-to-nearest (RTN, symmetric absmax scale per group) with GPTQ (Frantar et al., 2023) (symmetric) and AWQ (Lin et al., 2024) (asymmetric) as implemented in llm-compressor 0.14, both calibrated on 256 windows of 1024 tokens from each model's own training data.

The gains survive quantization. With GPTQ at int4, every perturbed student keeps its advantage over standard distillation quantized the same way. Adjacent blending's advantage even grows slightly, from 0.020 to 0.024 nats, and those of parallel blending and noise also grow slightly, to 0.017 and 0.015. With AWQ at int4 the advantages are essentially unchanged. So with either method the perturbed students quantize at least as well as the standard one. Plain round-to-nearest is mixed: at int4 the noise-trained student loses more than standard distillation, and the blended students lose less.

Round-to-nearest and weight kurtosis. At int3 under round-to-nearest the blended students are far more robust. Int3 raises the loss of standard distillation by 6.2 nats, but that of the two blends by only 1.2 and 1.5. The difference persists when we compare checkpoints from the middle of training

Table 2: **Zero-shot accuracy** (%, lm-evaluation-harness; normalized accuracy for HellaSwag, ARC, PIQA and SciQ, accuracy for Winogrande and LAMBADA). Each family at its best setting for that scale, with a perturbed stage of 30% (0.5B parallel blend 15%, 135M noise 50%). Δ Avg.: difference from standard distillation in the average over tasks, $\pm$ paired bootstrap standard error over examples. Best value per scale in bold.

	HellaSwag	ARC-E	ARC-C	PIQA	Wino.	LAMBADA	SciQ	Avg.	Δ Avg.
0.5B									
standard KD	45.0	57.9	29.0	68.0	52.7	37.5	77.3	52.5	–
noise σ=0.75	45.7	56.8	30.5	68.8	52.8	**39.1**	**81.3**	**53.6**	$+1.1 \pm 0.4$
parallel λ=0.5	45.5	**58.1**	30.3	68.3	**54.1**	36.7	77.5	52.9	$+0.5 \pm 0.3$
adjacent λ=0.5	**45.9**	58.0	**30.6**	**69.7**	53.4	36.3	80.9	53.5	$+1.1 \pm 0.3$
135M									
standard KD	30.1	44.4	25.2	60.1	**52.3**	21.3	64.0	42.5	–
noise σ=0.4	30.4	45.4	24.5	**61.3**	50.2	**22.6**	**65.1**	42.8	$+0.3 \pm 0.3$
parallel λ=0.7	**30.5**	46.1	25.1	60.3	50.0	22.1	62.6	42.4	-0.1 ± 0.4
adjacent λ=0.8	30.4	**46.8**	**25.3**	60.9	50.8	22.3	**65.1**	**43.1**	$+0.6 \pm 0.3$

at matched full-precision loss, and it tracks the weight distribution. The mean excess kurtosis of the linear-layer weights is about 0.5 for standard distillation and for noise, but 0.14 for both blends. Parallel blending at λ=0.7, which gives no gain at 0.5B, keeps the standard student's kurtosis and its round-to-nearest damage. Methods that compensate for quantization error remove most of the difference. With GPTQ the blended students' int3 damage is a quarter lower than standard distillation's, similar to noise, and with AWQ all students lose the same. The activation statistics we checked, the outlier ratio of the inputs to each linear layer and the fraction of very large residual-stream activations, do not differ between the students.

Downstream accuracy. Averaged over seven zero-shot tasks from lm-evaluation-harness, adjacent blending and noise both raise accuracy at 0.5B by about one point over standard distillation, from 52.5% to 53.5% and 53.6% (Table 2). Parallel blending gains half a point. The largest gains are on SciQ, about four points, and HellaSwag, about one, while both blends lose about a point on LAMBADA. At 135M the differences in the average are small, at most 0.6 points.

8 A SAMPLE-EFFICIENCY SEPARATION FOR TEACHER CORRECTION

In this section, we analyze the separation in data efficiency for the student between training on teacher-labeled corrupt data, versus statically-labeled corrupt data. Blending common inputs can produce an informative input that is rare in the original distribution. Recomputing the teacher's prediction on the blend then supplies information absent from the original targets. The following example makes this advantage precise under clean forward-KL loss, even when the static-target learner receives both original soft targets and all input metadata.

Setup. Fix $a > 0$ and $\rho \in (0, 1)$. Let

$$\Pr(X = 0) = \rho, \qquad \Pr(X = 1) = \Pr(X = -1) = \frac{1 - \rho}{2},$$

and consider the Bernoulli teacher and student family

$$p_\theta(x) = \sigma\big(\theta(1 - x^2)\big), \qquad \theta \in [-a, a], \qquad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

Here $p_\theta(x)$ denotes the probability of label 1. The common inputs ± 1 have prediction $1/2$, independent of θ, whereas the rare input 0 has prediction $\sigma(\theta)$.

Draw N independent pairs (X_i, X_i') of independent clean inputs and set $Z_i = (X_i + X_i')/2$. Both learners observe all (X_i, X_i', Z_i). The *static* learner additionally receives $p_\theta(X_i)$ and $p_\theta(X_i')$; the *corrected* learner instead receives $p_\theta(Z_i)$. Write their datasets as D_{s} and D_{c}, respectively. Both

protocols use N blended student examples and $2N$ clean input draws. Granting the static learner both original targets only strengthens that baseline.

Write

$$\mathrm{kl}(p, q) = p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q}$$

for Bernoulli KL divergence. For a predictor $q : \mathbb{R} \to (0, 1)$, define its clean risk and the minimax expected risk by

$$L_\theta(q) = \mathbb{E}_X \big[\mathrm{kl}(p_\theta(X), q(X))\big],$$
$$\mathcal{R}_b(N) = \inf_{\mathcal{A}} \sup_{\theta \in [-a,a]} \mathbb{E}\big[L_\theta(\mathcal{A}(D_b))\big], \qquad b \in \{\mathrm{s}, \mathrm{c}\},$$

where $\mathcal{A}$ ranges over all learning algorithms and the outer expectation includes the training data and any algorithmic randomness.

Theorem 8.1 (Training-efficiency separation)**.** *Let $C_a = \mathrm{kl}(\sigma(a), 1/2) > 0$. For every $N \geq 0$,*

$$\mathcal{R}_{\mathrm{s}}(N) = \rho C_a (1 - \rho)^{2N}, \tag{1}$$

$$\mathcal{R}_{\mathrm{c}}(N) = \rho C_a \left(\frac{(1 - \rho)^2}{2} \right)^N = 2^{-N} \mathcal{R}_{\mathrm{s}}(N). \tag{2}$$

These equalities also hold when the student is restricted to $p_{\widehat{\theta}}$ with $\widehat{\theta} \in [-a, a]$.

Proof. Let E_{s} be the event that none of the $2N$ original inputs equals 0, and let E_{c} be the event that every blend belongs to $\{-1, 1\}$. A blend is an endpoint exactly when both original inputs are that same endpoint. Hence

$$\Pr(E_{\mathrm{s}}) = (1 - \rho)^{2N}, \qquad \Pr(E_{\mathrm{c}}) = \left(\frac{(1 - \rho)^2}{2} \right)^N.$$

Outside E_{s}, an original target at 0 determines $\theta = \mathrm{logit}(p_\theta(0))$. Outside E_{c}, a corrected target at some $|Z_i| < 1$ determines

$$\theta = \frac{\mathrm{logit}(p_\theta(Z_i))}{1 - Z_i^2}, \qquad \mathrm{logit}(p) = \log \frac{p}{1 - p}.$$

Thus each learner can recover the teacher exactly outside its respective event. On that event, outputting $p_0 \equiv 1/2$ incurs clean risk at most ρC_a, since $p_\theta(\pm 1) = 1/2$ and $\mathrm{kl}(\sigma(\theta), 1/2) \leq C_a$ for $|\theta| \leq a$. This proves both upper bounds.

For the lower bounds, put equal prior probability on $\theta = a$ and $\theta = -a$. Conditional on either learner's event, all its observed targets are $1/2$, and its entire transcript is independent of θ. The posterior therefore remains uniform. Writing $p = \sigma(a)$, any predicted probability q at the clean input 0 satisfies

$$\tfrac{1}{2} \mathrm{kl}(p, q) + \tfrac{1}{2} \mathrm{kl}(1 - p, q) = C_a + \mathrm{kl}(1/2, q) \geq C_a.$$

The conditional Bayes clean risk is at least ρC_a. Multiplying by the respective event probabilities gives matching lower bounds, because worst-case risk is at least Bayes risk. The upper-bound predictors belong to the student family, proving the final assertion as well. □

Sample-complexity consequence. For any fixed $\alpha \in (0, 1)$, reaching expected clean risk at most $\alpha \rho C_a$ requires $\Theta(\rho^{-1})$ static examples but only $\Theta(1)$ corrected examples as $\rho \to 0$. This is a separation at fixed relative accuracy: the absolute target risk scales with ρ. The mechanism is that opposite common inputs blend to the informative input 0 with probability $(1 - \rho)^2/2$, while their original targets are both $1/2$.

9 Related work

Distillation in pretraining. Distillation into small pretrained language models is well established (Hinton et al., 2015; Peng et al., 2025). Progressive distillation from intermediate teacher checkpoints can induce an implicit curriculum (Panigrahi et al., 2025); a weaker first-stage teacher does

not reproduce our gain (Section 5). Work on language-model distillation has mostly changed the divergence or the sampling distribution, for example with on-policy or reverse-KL objectives (Gu et al., 2024; Agarwal et al., 2024). MiniPLM (Gu et al., 2025) uses the teacher to reweight the pretraining data, and Busbridge et al. (2025) give scaling laws for distillation. We keep the objective fixed and change the inputs on which it is evaluated.

Perturbations and derivative matching. Matching a teacher's derivatives, or its responses to perturbations, goes back to Jacobian and Sobolev training (Srinivas & Fleuret, 2018; Czarnecki et al., 2017). In vision, Beyer et al. (2022) show that teacher and student should see the same augmented view. Task-specific distillation of language models already trains on teacher-labeled augmented text, e.g. with word replacement (Jiao et al., 2020) or adversarially masked text (Rashid et al., 2021), and Liu et al. (2021) inject noise into both teacher and student in self-distillation for summarization. In vision, teacher-labeled augmentation is analyzed by Wang et al. (2022) and Stanton et al. (2021); Noisy Student (Xie et al., 2020) instead noises the student and labels the clean input. We study the pretraining setting and separate the teacher-side and student-side contributions. Unlike Jacobian matching, our students do not learn the teacher's local response to the perturbation.

Data-constrained pretraining and input augmentation. Repeating data for a few epochs is nearly as good as fresh data, with returns diminishing beyond about 16 epochs (Muennighoff et al., 2023; Chai & Xiong, 2026). Under repetition, dropout and much larger weight decay help (Xue et al., 2023; Kim et al., 2026), and so does corrupting the input while keeping the clean target: an auxiliary loss on masked inputs (Xu et al., 2026), random token replacement (Chen et al., 2026), or masking, which partly explains why diffusion language models tolerate more repetition (Ni et al., 2025; Prabhudesai et al., 2025). Kim et al. (2026) also distill an ensemble into a smaller student, by training on the ensemble's generated text. These methods perturb the student's input only; in distillation, we find that most of the benefit comes from letting the teacher predict on the perturbed input.

Mixing and merging inputs. Mixup-style interpolation (Zhang et al., 2018) has been applied to language-model distillation in embedding space (Liang et al., 2021). Merging neighboring tokens is used to save computation in vision transformers (Bolya et al., 2023) and in language-model pretraining (Peng, 2026), and embedding noise improves instruction fine-tuning (Jain et al., 2023). Our blends use related constructions as shared perturbations for distillation, and our ablation shows that their benefit depends on the teacher.

10 Conclusion

Distilling on perturbed inputs is a small change to standard distillation. Only the inputs of an early stage of training change, and the objective, the data and the token budget stay the same. It improved the student at both scales we tested, by 0.025 nats at 135M and 0.020 nats at 0.5B with the best variant, adjacent-token blending. In our experiments the perturbed stage worked best when it was short, 15–50% of training depending on how strongly the inputs are perturbed. For additive noise, the improvement comes mostly from the teacher's predictions on the perturbed inputs. This is why it mostly disappears without a teacher, and why perturbing only the student's input recovers only part of it.

An important limitation is the scale of our experiments: whether these gains persist for larger models or with substantially more training data remains open. The perturbations studied here provide a proof of concept that a teacher's predictions beyond ordinary training text can offer useful supervision. A promising direction for future work is to query the teacher on other inputs beyond standard text and the perturbations discussed here, and to explore how to choose those inputs to improve student learning.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-

generated mistakes. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.13649.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, et al. SmolLM2: When smol goes big — data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.

Lucas Beyer, Xiaohua Zhai, Amélie Royer, Larisa Markeeva, Rohan Anil, and Alexander Kolesnikov. Knowledge distillation: A good teacher is patient and consistent. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *International Conference on Learning Representations (ICLR)*, 2023.

Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, and Russ Webb. Distillation scaling laws. In *International Conference on Machine Learning*, 2025.

Yekun Chai and Haoyi Xiong. What is a repeated token worth? The scaling geometry of multi-epoch pretraining. *arXiv preprint arXiv:2610.05591*, 2026.

Michael K. Chen, Xikun Zhang, Fan Bai, Zhengding Hu, and Zhen Wang. Demystifying training-time augmentation for data-constrained language model pretraining. *arXiv preprint arXiv:2606.16246*, 2026.

Wojciech M. Czarnecki, Simon Osindero, Max Jaderberg, Grzegorz Swirszcz, and Razvan Pascanu. Sobolev training for neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

Gemma Team. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Yuxian Gu, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. MiniPLM: Knowledge distillation for pre-training language models. In *International Conference on Learning Representations*, 2025.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.

Neel Jain, Ping-yeh Chiang, Yuxin Wen, John Kirchenbauer, Hong-Min Chu, Gowthami Somepalli, Brian R Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. NEFTune: Noisy embeddings improve instruction finetuning. *arXiv preprint arXiv:2310.05914*, 2023.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.

Konwoo Kim, Suhas Kotha, Percy Liang, and Tatsunori Hashimoto. Pre-training under infinite compute. In *International Conference on Learning Representations*, 2026.

Kevin J. Liang, Weituo Hao, Dinghan Shen, Yufan Zhou, Weizhu Chen, Changyou Chen, and Lawrence Carin. MixKD: Towards efficient distillation of large-scale language models. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2011.00593.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Yang Liu, Sheng Shen, and Mirella Lapata. Noisy self-knowledge distillation for text summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 692–703, 2021.

Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems*, 2023.

Muse Glimmer Team. Muse Glimmer 30B: An on-device agentic model. *arXiv preprint arXiv:2610.24628*, 2026.

Jinjie Ni, Qian Liu, Longxu Dou, Chao Du, Zili Wang, Hang Yan, Tianyu Pang, and Michael Qizhe Shieh. Diffusion language models are super data learners. *arXiv preprint arXiv:2511.03276*, 2025.

Abhishek Panigrahi, Bingbin Liu, Sadhika Malladi, Andrej Risteski, and Surbhi Goel. Progressive distillation induces an implicit curriculum. In *International Conference on Learning Representations*, 2025.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.17557.

Bowen Peng. Efficient pre-training with token superposition. *arXiv preprint arXiv:2605.06546*, 2026.

Hao Peng, Xin Lv, Yushi Bai, Zijun Yao, Jiajie Zhang, Lei Hou, and Juanzi Li. Pre-training distillation for large language models: A design space exploration. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2410.16215.

Mihir Prabhudesai, Mengning Wu, Amir Zadeh, Katerina Fragkiadaki, and Deepak Pathak. Diffusion beats autoregressive in data-constrained settings. *arXiv preprint arXiv:2507.15857*, 2025.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

Ahmad Rashid, Vasileios Lioutas, and Mehdi Rezagholizadeh. MATE-KD: Masked adversarial TExt, a companion to knowledge distillation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

Suraj Srinivas and François Fleuret. Knowledge transfer with Jacobian matching. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. arXiv:1803.00443.

Samuel Stanton, Pavel Izmailov, Polina Kirichenko, Alexander A. Alemi, and Andrew Gordon Wilson. Does knowledge distillation really work? In *Advances in Neural Information Processing Systems*, 2021.

Huan Wang, Suhas Lohit, Michael Jones, and Yun Fu. What makes a "good" data augmentation in knowledge distillation – a statistical perspective. In *Advances in Neural Information Processing Systems*, 2022.

Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V. Le. Self-training with noisy student improves ImageNet classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.

Zhiwei Xu, Shihao Wu, Hanseul Cho, Wei Hu, and Yixin Wang. Data-constrained language model pretraining: Improved regularization and scaling laws. *arXiv preprint arXiv:2606.06888*, 2026.

Fuzhao Xue, Yao Fu, Wangchunshu Zhou, Zangwei Zheng, and Yang You. To repeat or not to repeat: Insights from scaling LLM under token-crisis. In *Advances in Neural Information Processing Systems*, 2023.

Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

A TRAINING DETAILS

Table 3: **Training configuration.** Both scales train the student from random initialization with a frozen teacher; the distillation loss is the forward KL divergence over the full vocabulary at temperature 1, with no next-token term.

	135M	0.5B
Student architecture	SmolLM2-135M	Qwen2.5-0.5B
Teacher (frozen)	SmolLM2-1.7B	Qwen2.5-1.5B
Vocabulary (shared)	49,152	151,936
Data	FineWeb-Edu, 58.6B-token pool	FineWeb-Edu, 65B-token pool
Steps $\times$ batch $\times$ length	$8000 \times 512 \times 1024$ (4.2B tokens)	$19073 \times 256 \times 2048$ (10B tokens)
Optimizer	AdamW, β=$(0.9, 0.95)$, weight decay 0.1	same
Peak learning rate	6×10^{-4}	1.2×10^{-3}
Schedule	WSD: 10% warmup, 10% decay	same
Precision	bfloat16	bfloat16
Compute per run	$\approx$20 H100-hours	$\approx$90 H100-hours

B DATA-CONSTRAINED RESULTS

Table 4: **Data-constrained pretraining** at 135M: final validation loss with 16 epochs over 262M unique tokens, by weight decay, against the same 4.2B-token budget without repetition (1 epoch). Two values: two initialization seeds. The 1-epoch runs at weight decay 1 use one seed. Noise: token noise σ=0.4 for the first 50% of training. Bold: the better of each objective with and without noise.

	1 epoch, wd		16 epochs, weight decay			
	0.1	1	0.1	0.3	1	3
Next-token prediction	3.1542	–	3.3365	–	3.2297	–
+ noise	**3.1516**	–	**3.3250**	–	**3.2214**	–
Standard distillation	3.1370	3.1318	3.2016	3.1812	3.1682 / 3.1693	3.2198
+ noise	**3.1177**	**3.1120**	**3.1693**	**3.1510**	**3.1448 / 3.1490**	–

C AUGMENTATION FROM CONCURRENT WORK

Recent work on data-constrained pretraining perturbs the student's input and keeps the clean target. Its most effective variant replaces a random 15% of tokens with random vocabulary items (Chen et al., 2026). We tried this replacement inside distillation in the same three ways, with one seed. Without a teacher it does not improve next-token training in our setting. With a teacher it behaves differently from our perturbations. Applied to the student's input only, with the teacher predicting on clean text, it gains 0.013 nats, while giving the replaced text to both models gains only 0.003. A replaced token is gone from the input, so the teacher's prediction on it is a prediction about different text, whereas our perturbations keep the true token. Student-side replacement still falls short of noise and adjacent blending on the same seed, which gain 0.019 and 0.025. We did not test the masked

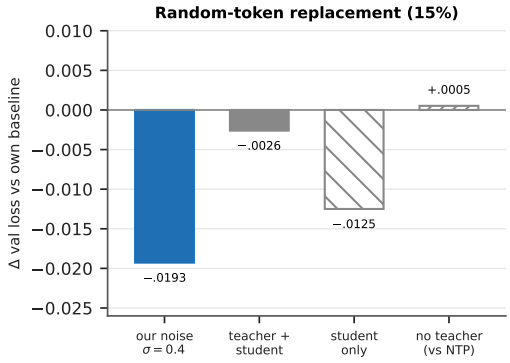


Figure 6: **Random-token replacement** (135M, one seed). Replacing 15% of tokens with random ones (Chen et al., 2026), given to both models, to the student only, or without a teacher, next to token noise on the same seed. Each arm is compared with its own baseline (standard distillation, or next-token prediction for the arm without a teacher).

auxiliary objective of Xu et al. (2026), which needs a second forward pass at every step. Fig. 6 shows the results.

D MECHANISM PROBES

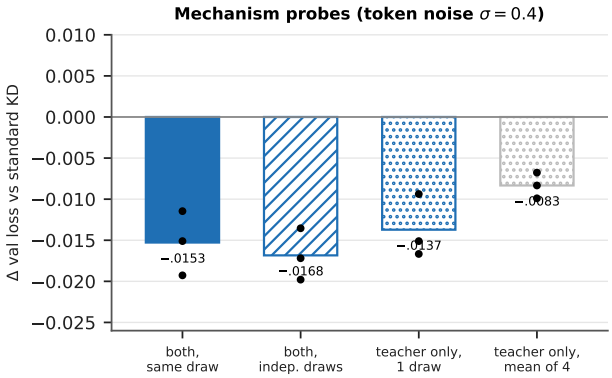


Figure 7: **Mechanism probes** (135M, token noise σ=0.4 for the first 50% of training; difference from standard distillation, mean over seeds with individual seeds as dots).

Shared or independent draws. Drawing the teacher's and the student's noise independently gives the same gain as sharing it (-0.0168 against -0.0153, three seeds; Fig. 7). In line with this, the students do not learn how the teacher's prediction responds to the perturbation. We measure each model's response as the change in its next-token distribution under a small step along the noise direction. For every student we trained, this response is nearly uncorrelated with the teacher's (correlation at most about 0.03), with or without noise in training. *One draw or the average of four.* Averaging the teacher's predictions over four noise draws into one target, which has the same expected value as a single draw, gives a smaller gain than a single draw given to the teacher only (-0.0076 against -0.0122 on the same two seeds). This suggests that the randomness of the target helps, not only its expected value. The differences in this appendix are comparable to the spread between seeds, and we regard them as indicative.